Lecture L13

*Blind review — please score this lecture on row **L13** in the Excel form (1–10 per dimension).*

Functional Dependencies, Normalization, and Relational Schema Design (1NF through BCNF)

Learning Objectives

By the end of this lecture, you will be able to:

1. **Remember:** Define functional dependency, superkey, candidate key, prime attribute, and the four normal forms (1NF, 2NF, 3NF, BCNF) precisely.
2. **Understand:** Explain why unnormalized schemas cause insertion, deletion, and update anomalies using concrete examples.
3. **Apply:** Given a relation schema and a set of functional dependencies, determine its normal form (up to BCNF) and decompose it to achieve a higher normal form while preserving dependencies and ensuring a lossless join.
4. **Analyze:** Trace a transitive dependency through a nonprime attribute and predict the anomalies it would cause.
5. **Evaluate:** Compare the trade-offs between 3NF and BCNF and decide which normal form is appropriate for a given design.

---

1. The Cost of Poor Schema Design

Every database you've ever used that felt sluggish, inconsistent, or strangely resistant to new data had one root cause: the schema was lying. Not maliciously — but by forcing multiple independent facts about the world to share the same row, it created a structure where some truths could only be expressed alongside others. This is the central tension of schema design: we want to record facts independently, but a single table forces them into artificial cohabitation.

Before we define formal rules, consider a single table that tracks employee project assignments:

```
EMP_PROJ(SSN, PNumber, EName, PLocation, Hours)
```

Suppose we store these facts:

SSN	PNumber	EName	PLocation	Hours
123	P01	Alice	Boston	10
123	P02	Alice	Denver	20
456	P01	Bob	Boston	15

Three anomalies immediately appear:

- **Update anomaly:** If Alice changes her name, we must update every row where she appears. Missing one row leaves the database inconsistent — some rows will say Alice, others will still say her old name. The database now asserts two contradictory names for the same employee.

- **Insertion anomaly:** We cannot record a new project's location (e.g., P03 in Chicago) until an employee is assigned to it, because the key {SSN, PNumber} demands an SSN value. The very act of storing a fact about a project is gated by the existence of an unrelated fact about an employee.
- **Deletion anomaly:** If Bob leaves the company and we delete his only row, we lose the fact that project P01 is located in Boston. Removing an employee destroys project information that should have survived independently.

These anomalies arise because the table crams together facts about employees, projects, and assignments into one relation. The table is telling a single story with three characters, but each character's identity is entangled with the others. **Normalization** uses functional dependencies to systematically split such a table into smaller, anomaly-free relations — each relation tells one story about one kind of thing.

A Cross-Domain Perspective: Normalization as Information Independence

The problem we just diagnosed is not unique to databases. It is structurally identical to a problem in **programming language design**: when a single module encodes multiple orthogonal concerns, you get what software engineers call *tight coupling*. A class that mixes authentication logic with payment processing forces you to deploy both concerns together, test them together, and break them together. Normalization is, in this light, the database equivalent of the **Single Responsibility Principle** — each relation should encapsulate exactly one fact about one kind of entity, just as each class should have exactly one reason to change.

The formal machinery we are about to build (functional dependencies, closures, decomposition algorithms) is the database theorist's answer to a question that appears across computing: *given a set of constraints, how do we partition a structure so that each partition is independently maintainable?* The same mathematical skeleton appears in:

- **Compiler design:** When a parser decomposes a monolithic grammar into independent production rules, each rule captures one syntactic fact. Anomalies in grammars (ambiguity, left recursion) are the compiler's version of update anomalies.
- **Distributed systems:** When you shard a database across nodes, you are performing a physical decomposition driven by access patterns. The logical decomposition we study here is the precursor — get the logical grouping wrong, and no physical sharding strategy can save you from cross-node anomalies.
- **Data warehousing and dimensional modeling:** Star schemas deliberately *denormalize* dimension tables (e.g., storing customer city alongside customer name) to optimize read performance. But this denormalization is a conscious, calculated violation of normalization principles — and the warehouse designer must explicitly manage the resulting update anomalies through ETL processes. You cannot intelligently denormalize what you do not understand in normalized form.

This last point deserves emphasis: **NoSQL document stores, key-value stores, and graph databases all make different trade-offs about normalization.** MongoDB encourages embedding related data in a single document — which is exactly the EMP_PROJ design we just condemned. Why is that sometimes acceptable? Because a document store assumes a different access pattern (read-heavy, aggregate-oriented) and a different consistency model (eventual consistency, no multi-document transactions). The anomalies still exist; the database simply shifts the burden of managing them from the DBMS to the application developer. Understanding normalization lets you make that trade-off explicitly rather than accidentally.

---

2. Foundational Concepts: Functional Dependencies and Keys

2.1 Functional Dependencies

Intuition. A functional dependency (FD) is a rule about the real world that states: "If I know the values of attributes X, then I can uniquely determine the values of attributes Y." But here's the subtlety that trips up most newcomers: an FD is not a statement about your current data. It's a statement about the *universe of possible data* your schema must accommodate.

Think of an FD like a physical law, not a weather report. Newton's second law doesn't say "in my lab today, F=ma happened to hold." It says "in any physically possible situation, force equals mass times acceleration." Similarly, SSN $\rightarrow$ EName doesn't say "in our current table, each SSN maps to one name." It says "in any legitimate state of this enterprise, an SSN will identify exactly one employee name — now, tomorrow, and after a million updates."

An original analogy: FDs as invariants in a state machine. Imagine your database as a state machine whose states are all legal instances of your schema. An FD $X \rightarrow Y$ is an *invariant* of that machine: a predicate that must hold in every reachable state. When you design a schema, you are not just drawing boxes; you are declaring which invariants the system must preserve across all transactions. A transaction that violates an FD is not merely "bad practice" — it is a transition to an illegal state, exactly as forbidden as dividing by zero in a arithmetic expression. This perspective connects database theory to **formal methods** (where invariants are the backbone of program correctness) and explains why FDs are constraints, not observations: constraints define the legal state space; observations merely sample it.

Formal definition. Let R be a relation schema, and let $X \subseteq R$ and $Y \subseteq R$ be sets of attributes. An FD $X \rightarrow Y$ holds on R if, for every possible legal instance $r(R)$, whenever two tuples agree on X, they must also agree on Y:

$$\forall t_1, t_2 \in r : \quad t_1[X] = t_2[X] \implies t_1[Y] = t_2[Y]$$

We say X *functionally determines* Y, or Y is *functionally dependent* on X.

Trivial vs. nontrivial FDs. An FD $X \rightarrow Y$ is *trivial* if $Y \subseteq X$ (e.g., {SSN, EName} $\rightarrow$ {EName}). Trivial FDs always hold by definition and provide no design constraint. Nontrivial FDs ($Y \not\subseteq X$) encode real-world constraints and drive normalization.

Closure of a set of FDs. Given a set F of FDs, the *closure* F^+ is the set of all FDs logically implied by F. The Armstrong axioms (reflexivity, augmentation, transitivity) are sound and complete for computing F^+. In practice, we compute the *attribute closure* X^+ (all attributes determined by X under F) to test whether an FD $X \rightarrow Y$ is in F^+: it is if and only if $Y \subseteq X^+$.

2.2 Keys and Superkeys

- A **superkey** of R is a set of attributes $K \subseteq R$ such that $K \rightarrow R$ holds (i.e., K functionally determines every attribute in the schema).
- A **candidate key** is a minimal superkey: no proper subset of it is a superkey.
- A **primary key** is one candidate key chosen by the designer for implementation purposes.
- A **prime attribute** is any attribute that belongs to at least one candidate key. A **nonprime attribute** belongs to no candidate key.

---

3. First Normal Form (1NF): The Atomicity Baseline

Definition. A relation schema R is in 1NF if the domain of each attribute contains only *atomic* (indivisible) values, and each tuple contains exactly one value from that domain per attribute. In relational terms: no repeating groups, no nested relations, no multivalued attributes.

What 1NF forbids (concrete example). Consider a course enrollment table that violates 1NF:

`STUDENT_COURSES(StudentID, Name, Courses)`

where `Courses` is a set-valued attribute:

StudentID	Name	Courses
S01	Alice	{CS101, MA201}
S02	Bob	{CS101, PH301, CS202}

This violates 1NF because the `Courses` attribute is not atomic — it contains a set of values. The relational model requires us to flatten this into:

`ENROLLMENT(StudentID, CourseID)`

with one row per student-course pair.

Practical relevance: why 1NF matters in real systems. The 1NF requirement is not an academic exercise — it directly determines whether you can use SQL's core operators effectively. Consider:

- **Indexing:** If `Courses` is a set, you cannot create a standard B-tree index on individual course codes. Querying "all students taking CS101" requires a full table scan with application-level parsing rather than an index seek.
- **Joins:** You cannot join `STUDENT_COURSES` with a `COURSE` table on course code because the join condition operates on atomic values. The DBMS's query optimizer — which relies on statistics about value distributions — is blind to the internal structure of a multivalued attribute.
- **Referential integrity:** Foreign key constraints enforce that a value in one column exists in another column. If `Courses` contains multiple values, the DBMS cannot enforce that each individual course code references a valid row in the `COURSE` table.

Industry case study: the JSON column trap in PostgreSQL. Modern RDBMSs like PostgreSQL support JSON/JSONB columns that can store nested structures. A common anti-pattern is:

```
CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    customer_name TEXT,
    line_items JSONB  -- e.g., [{"sku": "A", "qty": 2}, {"sku": "B", "qty": 1}]
);
```

This technically satisfies 1NF (JSONB is an atomic scalar type from PostgreSQL's perspective), but it *logically* violates 1NF because the `line_items` field encodes a repeating group. The practical consequences are identical to the 1NF violation above:

- To find all orders containing SKU "A", you must write: `WHERE line_items @> '[{"sku": "A"}]'` — a full-table scan with JSON path evaluation, not an index lookup on `sku`.
- You cannot declare a foreign key from `line_items.sku` to a `products` table. The referential integrity guarantee is lost; application code must validate SKUs.

- Updating the quantity of a specific line item requires extracting, modifying, and rewriting the entire JSONB document — an anomaly structurally identical to the update anomaly in our original EMP_PROJ table.

The lesson: **1NF is not about the physical storage type; it is about whether each attribute-value pair represents exactly one fact.** Storing a JSON array in a column may satisfy the letter of 1NF but violates its spirit, and you pay the same anomaly costs.

---

4. Second Normal Form (2NF): Eliminating Partial Dependencies

Motivation. 1NF eliminated repeating groups, but it did nothing about FDs that involve only *part* of a candidate key. These partial dependencies are the source of redundancy in 1NF schemas.

Definition. A relation schema R is in 2NF if it is in 1NF and every nonprime attribute is fully functionally dependent on *every* candidate key (i.e., no nonprime attribute is functionally dependent on a proper subset of any candidate key).

Equivalently: For every nonprime attribute A and every candidate key K, there must be no FD $X \to A$ where $X \subset K$ (a proper subset of K).

Concrete example: the EMP_PROJ table revisited. Our EMP_PROJ table has the single candidate key {SSN, PNumber}. The nonprime attributes are EName, PLocation, and Hours. We observe:

- SSN → EName (a partial dependency: SSN is a proper subset of the key, and EName is nonprime).
- PNumber → PLocation (another partial dependency: PNumber is a proper subset of the key, and PLocation is nonprime).
- {SSN, PNumber} → Hours (full dependency: Hours depends on the entire key — this is the assignment fact).

Because partial dependencies exist, EMP_PROJ is not in 2NF. The redundancy: an employee's name is repeated for every project they work on; a project's location is repeated for every employee assigned to it. The decomposition to achieve 2NF splits on the partial dependencies:

```
EMP(SSN, EName)           -- key: {SSN}; FD: SSN → EName
PROJ(PNumber, PLocation)  -- key: {PNumber}; FD: PNumber → PLocation
ASSIGN(SSN, PNumber, Hours) -- key: {SSN, PNumber}; FD: {SSN, PNumber} → Hours
```

Practical relevance: 2NF in financial systems design. Consider a **securities trading system** that records trade executions:

```
TRADE(TradeID, SecurityID, TraderID, SecuritySymbol, TraderName, Quantity, Price)
```

Candidate key: {TradeID} (each trade has a unique identifier). Assume the business rules: - SecurityID → SecuritySymbol (a security's symbol is determined by its ID). - TraderID → TraderName (a trader's name is determined by their ID).

Both are partial dependencies on the sole candidate key? No — the key is {TradeID}, and neither SecurityID nor TraderID is a subset of {TradeID}. So the table is in 2NF, and no decomposition is required by 2NF alone.

But now consider a variant where the business uses a **composite key**:

```
TRADE_COMPOSITE(TradeDate, SequenceNum, SecurityID, TraderID, SecuritySymbol, TraderName, Quantit
```

Candidate key: {TradeDate, SequenceNum} (trades are uniquely identified by date plus a within-day sequence number). Now: - SecurityID → SecuritySymbol (SecurityID is not part of the key — this is not a partial dependency; it's a full dependency on a non-key attribute, which we'll address in 3NF). - But suppose the system also enforces TraderID → TraderName and SecurityID → SecuritySymbol.

Are these partial dependencies? Only if SecurityID or TraderID is a proper subset of the candidate key. They are not. So this table is in 2NF. The redundancy from SecurityID → SecuritySymbol exists but is not a 2NF violation — it's a 3NF violation (transitive dependency).

The real-world lesson: In practice, 2NF violations most commonly arise when a designer chooses a composite natural key (e.g., OrderID + LineItemNum) and then includes attributes that depend on only one component (e.g., OrderDate depending on OrderID alone). Systems that use surrogate keys (auto-increment integers) for every table often bypass 2NF violations entirely because there are no proper subsets of a single-attribute key. But they still face 3NF violations. This is why experienced practitioners often say "2NF is rarely an issue if you use surrogate keys" — but they understand *why* that's true, not just that it's a rule of thumb.

---

5. Third Normal Form (3NF): Eliminating Transitive Dependencies

Motivation. Even in 2NF, a nonprime attribute can depend on another nonprime attribute, creating a *transitive* chain from the key to the first nonprime to the second nonprime. This transitive dependency causes the same redundancy and anomalies we saw earlier, just one step removed from the key.

Definition. A relation schema R is in 3NF if, for every nontrivial FD $X \to A$ (where A is a single attribute not in X), at least one of the following holds: 1. X is a superkey of R, or 2. A is a prime attribute (belongs to some candidate key).

Equivalently: No nonprime attribute is transitively dependent on any candidate key.

Classic example: EMP_DEPT. Consider:

```
EMP_DEPT(SSN, EName, BDate, Address, DNumber, DName, DMgrSSN)
```

with candidate key {SSN} and FDs: - SSN → {EName, BDate, Address, DNumber} (full dependency on the key). - DNumber → {DName, DMgrSSN} (a department determines its name and manager).

This creates a transitive dependency: SSN → DNumber → DName (and similarly to DMgrSSN). DName is a nonprime attribute dependent on DNumber, which is not a superkey. The table is not in 3NF. The decomposition:

```
EMP(SSN, EName, BDate, Address, DNumber)  -- key: {SSN}
DEPT(DNumber, DName, DMgrSSN)              -- key: {DNumber}
```

Practical relevance: 3NF in healthcare information systems. Consider a **hospital patient management system**:

```
PATIENT_VISIT(VisitID, PatientID, PatientName, PatientDOB,
              WardID, WardName, WardChargeNurse,
              AttendingPhysID, PhysicianName, PhysicianSpecialty)
```

Candidate key: {VisitID} (each hospital visit has a unique identifier). The FDs reflect the domain: - PatientID → {PatientName, PatientDOB} - WardID → {WardName, WardChargeNurse} - AttendingPhysID → {PhysicianName, PhysicianSpecialty}

This table is in 2NF (no partial dependencies on the single-attribute key) but not in 3NF. The transitive dependencies are: - VisitID → PatientID → {PatientName, PatientDOB} - $\text{VisitID